

Technical Requirement Document

Here is the Technical Requirement Document (TRD) based on the provided Functional Requirement Document (FRD):

TR-DLT-001: Load Customer and Order Data into Delta Tables

Related Functional Requirement(s): FR-DLT-001

Objective: Implement a PySpark job to read customer and order CSV data and load it into Delta tables.

Target Cluster Configuration:

- Cluster Name: Data Processing Cluster
- Databricks Runtime Version: 10.4.x-scala2.12
- Node Type: Standard_DS3_v2
- Driver Node: Standard_DS3_v2
- Worker Nodes: 2-4 Standard_DS3_v2
- Autoscaling: Enabled
- Auto Termination: 30 minutes
- Libraries Installed: delta-lake, spark-csv

Source Data Details:

- Dataset Name: customerdata
- Location (Path/Table): /Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/customerdata
- Format: CSV
- Description: Customer data in CSV format
- Dataset Name: orderdata
- Location (Path/Table): /Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/orderdata
- Format: CSV
- Description: Order data in CSV format

Target Data Details:

- Output Name: customer
- Location: Delta table
- Format: Delta
- Description: Customer data in Delta format
- Output Name: order

- Location: Delta table
- Format: Delta
- Description: Order data in Delta format

Job Flow / Pipeline Stages:

- Read customer CSV data into a Delta table named "customer"
- Read order CSV data into a Delta table named "order"
- Validate schema of customer and order tables

Data Transformations / Business Logic:

- Step: Read CSV data into Delta tables
- Description: Read customer and order CSV data into Delta tables
- Transformation Logic: Use PySpark to read CSV data and write to Delta tables

Error Handling and Logging:

- Log errors and exceptions to a logging table
- Handle schema validation errors and data loading errors

TR-DLT-002: Transform Order Data by Adding TotalAmount Column

Related Functional Requirement(s): FR-DLT-002

Objective: Implement a PySpark job to add a new column "TotalAmount" to the order table.

Target Cluster Configuration: Same as TR-DLT-001

Source Data Details:

- Dataset Name: order
- Location (Path/Table): Delta table
- Format: Delta
- Description: Order data in Delta format

Target Data Details:

- Output Name: order
- Location: Delta table
- Format: Delta
- Description: Order data with TotalAmount column in Delta format

Job Flow / Pipeline Stages:

- Add a new column "TotalAmount" to the order table
- Calculate the value of TotalAmount as the product of PricePerUnit and Qty

Data Transformations / Business Logic:

- Step: Add TotalAmount column to order table
- Description: Add a new column "TotalAmount" to the order table
- Transformation Logic: Use PySpark to add a new column and calculate its value

Error Handling and Logging:

- Log errors and exceptions to a logging table
- Handle data transformation errors

TR-DLT-003: Cleanse Customer and Order Data

Related Functional Requirement(s): FR-DLT-003

Objective: Implement a PySpark job to remove null and duplicate records from customer and order tables.

Target Cluster Configuration: Same as TR-DLT-001

Source Data Details:

- Dataset Name: customer
- Location (Path/Table): Delta table
- Format: Delta
- Description: Customer data in Delta format
- Dataset Name: order
- Location (Path/Table): Delta table
- Format: Delta
- Description: Order data in Delta format

Target Data Details:

- Output Name: customer
- Location: Delta table
- Format: Delta
- Description: Cleaned customer data in Delta format
- Output Name: order
- Location: Delta table
- Format: Delta
- Description: Cleaned order data in Delta format

Job Flow / Pipeline Stages:

- Remove rows with null values from customer and order tables
- Remove duplicate rows from customer and order tables

Data Transformations / Business Logic:

- Step: Cleanse customer and order data
- Description: Remove null and duplicate records from customer and order tables
- Transformation Logic: Use PySpark to remove null and duplicate records

Error Handling and Logging:

- Log errors and exceptions to a logging table
- Handle data cleansing errors

TR-DLT-004: Create ordersummary Table and Load Joined Data**Related Functional Requirement(s): FR-DLT-004**

Objective: Implement a PySpark job to create an ordersummary table and load joined data from customer and order tables.

Target Cluster Configuration: Same as TR-DLT-001

Source Data Details:

- Dataset Name: customer
- Location (Path/Table): Delta table
- Format: Delta
- Description: Customer data in Delta format
- Dataset Name: order
- Location (Path/Table): Delta table
- Format: Delta
- Description: Order data in Delta format

Target Data Details:

- Output Name: ordersummary
- Location: Delta table
- Format: Delta
- Description: Joined customer and order data in Delta format

Job Flow / Pipeline Stages:

- Create ordersummary table if it does not exist
- Join customer and order tables on CustId
- Load joined data into ordersummary table as an SCD type 2 table

Data Transformations / Business Logic:

- Step: Join customer and order data
- Description: Join customer and order tables on CustId
- Transformation Logic: Use PySpark to join customer and order tables

Error Handling and Logging:

- Log errors and exceptions to a logging table
- Handle data loading errors

TR-DLT-005: Update ordersummary SCD Type 2 Table on Customer Changes**Related Functional Requirement(s): FR-DLT-005**

Objective: Implement a PySpark job to update the ordersummary SCD type 2 table on customer changes.

Target Cluster Configuration: Same as TR-DLT-001

Source Data Details:

- Dataset Name: customer
- Location (Path/Table): Delta table
- Format: Delta
- Description: Customer data in Delta format

Target Data Details:

- Output Name: ordersummary
- Location: Delta table
- Format: Delta
- Description: Updated ordersummary data in Delta format

Job Flow / Pipeline Stages:

- Detect changes in customer table
- Update ordersummary table by making old records inactive and new records active
- Update StartDate and EndDate accordingly to maintain history

Data Transformations / Business Logic:

- Step: Update ordersummary table on customer changes
- Description: Update ordersummary table on customer changes
- Transformation Logic: Use PySpark to update ordersummary table

Error Handling and Logging:

- Log errors and exceptions to a logging table
- Handle data update errors

TR-DLT-006: Create customeraggregatespend Table and Load Aggregated Data

Related Functional Requirement(s): FR-DLT-006

Objective: Implement a PySpark job to create a customeraggregatespend table and load aggregated data from ordersummary table.

Target Cluster Configuration: Same as TR-DLT-001

Source Data Details:

- Dataset Name: ordersummary
- Location (Path/Table): Delta table
- Format: Delta
- Description: Joined customer and order data in Delta format

Target Data Details:

- Output Name: customeraggregatespend
- Location: Delta table
- Format: Delta
- Description: Aggregated customer spend data in Delta format

Job Flow / Pipeline Stages:

- Create customeraggregatespend table if it does not exist
- Aggregate TotalAmount from ordersummary table by Name and Date
- Load aggregated data into customeraggregatespend table

Data Transformations / Business Logic:

- Step: Aggregate customer spend data
- Description: Aggregate TotalAmount from ordersummary table by Name and Date
- Transformation Logic: Use PySpark to aggregate customer spend data

Error Handling and Logging:

- Log errors and exceptions to a logging table
- Handle data aggregation errors

TR-DLT-007: Implement Data Processing using Delta Live Tables

Related Functional Requirement(s): FR-DLT-007

Objective: Implement a Delta Live Tables pipeline to perform the required data transformations and loading.

Target Cluster Configuration: Same as TR-DLT-001

Job Flow / Pipeline Stages:

- Implement the data processing pipeline using Delta Live Tables

Data Transformations / Business Logic:

- Step: Implement Delta Live Tables pipeline
- Description: Implement the data processing pipeline using Delta Live Tables
- Transformation Logic: Use Delta Live Tables to perform data transformations and loading

Error Handling and Logging:

- Log errors and exceptions to a logging table
- Handle pipeline execution errors